Retrieval-Augmented Generation (RAG)

How to make an LLM answer from your own documents, from a 40-line prototype to hybrid search, reranking, evaluation, agents and production.

From the basics to advanced techniques, with explanations and working code.

1. **Foundations**: why RAG exists, the pipeline, a complete RAG system from scratch
2. **Building blocks**: loading, chunking, embedding and storing, grounded prompts, a full OpenAI + Chroma app
3. **Improving retrieval**: evaluation first, the vocabulary mismatch problem, hybrid search, reranking, query rewriting, HyDE
4. **Advanced and production**: evaluating answers, LangChain, agentic RAG, GraphRAG, security and deployment

Contents

Part 1: Foundations

1. Why RAG?
2. The RAG pipeline
3. A complete RAG system from scratch

Part 2: Building blocks

4. Loading documents
5. Chunking
6. Embedding and storing
7. Prompting for grounded answers
8. A full RAG app: OpenAI + Chroma

Part 3: Improving retrieval

9. Measure first: a retrieval test set
10. The vocabulary mismatch problem
11. Hybrid search
12. Reranking with a cross-encoder
13. Query rewriting
14. HyDE: search with a hypothetical answer
15. Other retrieval techniques

Part 4: Advanced and production

16. Evaluating the answers
17. RAG with LangChain
18. Agentic RAG
19. Other advanced patterns
20. Production checklist

Summary and interview questions

Cheat sheet

Interview questions to practise

Part 1: Foundations

1. Why RAG?

Large language models are trained on a fixed snapshot of public data. On their own they:

- **don't know your private data**: your company's policies, contracts, tickets or codebase;
- **don't know recent events** after their training cutoff;
- **hallucinate**: when they don't know, they often produce a confident, plausible and wrong answer;
- **can't cite sources**, so answers are hard to verify.

Retrieval-Augmented Generation fixes this by *looking things up first*. Before the model answers, the system retrieves relevant passages from your documents and puts them in the prompt. The model then answers *from that evidence*, and can cite it.

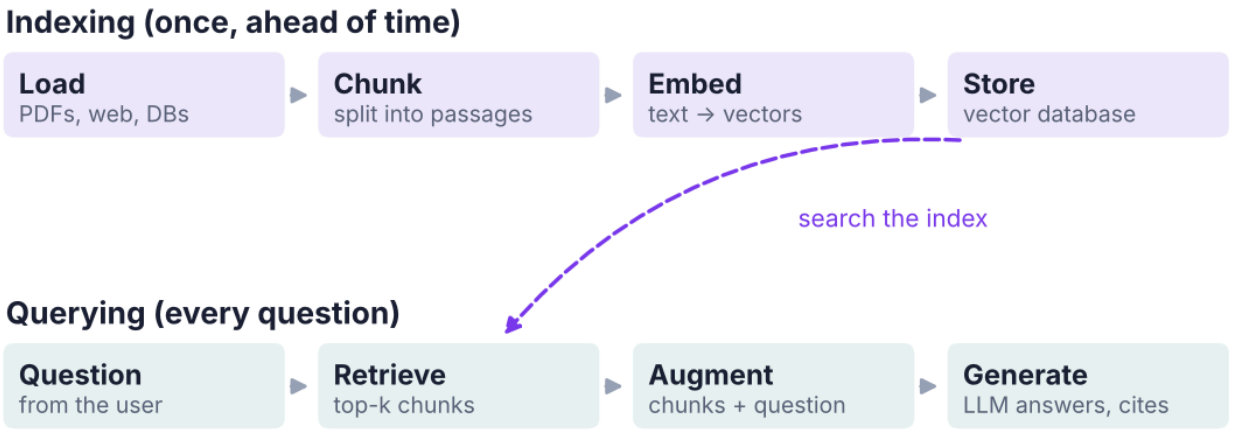
> **In plain words:** RAG turns a closed-book exam into an open-book exam. The model is still the student, but now it can look up the relevant pages before answering.

RAG or fine-tuning?

Need	Best approach
Answer questions using specific, changing knowledge	**RAG**: update the documents and the answers change immediately
Cite sources and show evidence	**RAG**
Restrict answers by user permissions	**RAG**: filter what each user can retrieve
Change the model's style, format or behaviour	**Fine-tuning**
Teach a specialised skill or output format	**Fine-tuning**
Both	Fine-tune for behaviour, RAG for knowledge; they combine well

2. The RAG pipeline

RAG has two phases. **Indexing** happens once, ahead of time (and again whenever documents change). **Querying** happens for every question.



Indexing turns documents into searchable vectors once. At query time, the question is used to retrieve the most relevant chunks, which are added to the prompt.

3. A complete RAG system from scratch

The best way to understand RAG is to build it with no frameworks. Our knowledge base is the help centre of **Brightwave**, a fictional broadband company. Each document is one help article.

Python

Tested: output below is real

```python
knowledge_base = [
    "Refunds: you can request a full refund within 30 days of purchase. Refunds are paid to the original payment method within 5 working days.",
    "Cancelling your contract: give 30 days notice by phone or in the app. Early cancellation fees apply during the first 12 months.",
    "Error E-4021 means the router cannot reach the network. Restart the router and check that the fibre cable is firmly connected.",
    "Error E-1007 means your account is suspended because a payment failed. Update your card details in the app to restore service.",
    "Slow internet speeds are often caused by Wi-Fi interference. Move the router away from microwaves and thick walls, or use a cable.",
    "Moving house: we can transfer your broadband to a new address. Tell us at least 14 days before you move to avoid a gap in service.",
    "Our Fibre 500 plan costs 45 euro per month with a 12 month contract. Fibre 1000 costs 60 euro per month.",
    "Customer support is open 8am to 10pm, seven days a week. Call 1800 555 0199 or use live chat in the app.",
    "Parental controls let you block websites and set screen time limits for each device from the app settings.",
    "Bills are issued on the 1st of every month. Pay by direct debit to get a 2 euro monthly discount.",
    "If your router is lost or damaged, a replacement router costs 50 euro and arrives within 3 working days.",
    "Installation: an engineer visits within 10 working days of your order to connect the fibre line to your home.",
]
print(len(knowledge_base), "documents")
```

Output

```
12 documents
```

Step 1: a retriever (BM25)

BM25 is the classic keyword-search algorithm behind Elasticsearch and most search engines. It scores a document higher when it contains the query's words, especially *rare* words (like an error code), and it adjusts for document length so long documents don't win just by being long.

Python

Tested: output below is real

```python
import re
import numpy as np

STOPWORDS = set("a an the to of in and or is are be by for on at my i you your we our "
                "how do what does when will get it me can".split())

def tokenize(text):
    words = re.findall(r"[a-z0-9]+(?:-[a-z0-9]+)*", text.lower())   # keeps codes like e-4021 whole
    return [w for w in words if w not in STOPWORDS]

class BM25:
    def __init__(self, docs, k1=1.5, b=0.75):
        self.docs = [tokenize(d) for d in docs]
        self.k1, self.b = k1, b
        self.avg_len = np.mean([len(d) for d in self.docs])
        n_docs = len(self.docs)
        vocab = {w for d in self.docs for w in d}
        doc_freq = {w: sum(w in d for d in self.docs) for w in vocab}
        self.idf = {w: np.log((n_docs - df + 0.5) / (df + 0.5) + 1) for w, df in doc_freq.items()}

    def scores(self, query):
        result = np.zeros(len(self.docs))
        for word in set(tokenize(query)):
            if word not in self.idf:
                continue
            for i, doc in enumerate(self.docs):
                tf = doc.count(word)
                norm = 1 - self.b + self.b * len(doc) / self.avg_len
                result[i] += self.idf[word] * tf * (self.k1 + 1) / (tf + self.k1 * norm)
        return result

    def search(self, query, k=3):
        s = self.scores(query)
        top = np.argsort(-s)[:k]
        return [(int(i), round(float(s[i]), 2)) for i in top]

retriever = BM25(knowledge_base)
for q in ["what does error E-4021 mean", "how much is fibre 1000"]:
    print(q, "->", retriever.search(q))
```

Output

```
what does error E-4021 mean -> [(2, 3.82), (3, 1.7), (1, 0.0)]
how much is fibre 1000 -> [(6, 3.66), (11, 1.45), (2, 1.32)]
```

Step 2: augment, building a grounded prompt

The retrieved chunks go into the prompt, numbered so the model can cite them. The instructions tell the model to use *only* this context, and to say so when the answer isn't there.

Python — Tested: output below is real

```python
def build_prompt(question, k=3):
    hits = retriever.search(question, k)
    context = "\n".join(f"[{n}] {knowledge_base[i]}" for n, (i, _) in enumerate(hits, start=1))
    return f"""You are Brightwave's support assistant.
Answer the question using ONLY the sources below. Cite sources like [1].
If the sources don't contain the answer, say "I don't know" and suggest contacting support.

Sources:
{context}

Question: {question}
Answer:"""

print(build_prompt("My router shows E-4021, what should I do?"))
```

Output

```
You are Brightwave's support assistant.
Answer the question using ONLY the sources below. Cite sources like [1].
If the sources don't contain the answer, say "I don't know" and suggest contacting support.

Sources:
[1] Error E-4021 means the router cannot reach the network. Restart the router and check that the fibre cable is firmly connected.
[2] If your router is lost or damaged, a replacement router costs 50 euro and arrives within 3 working days.
[3] Slow internet speeds are often caused by Wi-Fi interference. Move the router away from microwaves and thick walls, or use a cable.

Question: My router shows E-4021, what should I do?
Answer:
```

Step 3: generate

Python — Needs: pip install openai

```python
from openai import OpenAI

client = OpenAI()   # reads OPENAI_API_KEY

def answer(question):
    response = client.chat.completions.create(
        model="gpt-4.1",
        temperature=0,
        messages=[{"role": "user", "content": build_prompt(question)}],
    )
    return response.choices[0].message.content

print(answer("My router shows E-4021, what should I do?"))
# "Error E-4021 means your router cannot reach the network. Restart the router and
#  check that the fibre cable is firmly connected [1]."
print(answer("Do you offer satellite TV?"))
# "I don't know. The sources don't mention satellite TV; please contact support ..."
```

That's a complete RAG system in about 40 lines. Everything else in this guide makes one of these three steps better.

Part 2: Building blocks

4. Loading documents

Real knowledge lives in PDFs, Word files, web pages, wikis and databases. Whatever the source, convert it to plain text and keep **metadata**: the source file, page number, title, date and access permissions. You'll need it for citations and filtering.

Python — Needs: pip install pypdf

```python
from pypdf import PdfReader

def load_pdf(path):
    pages = []
    for number, page in enumerate(PdfReader(path).pages, start=1):
        text = page.extract_text() or ""
        if text.strip():
            pages.append({"text": text, "source": path, "page": number})
    return pages

documents = load_pdf("employee_handbook.pdf")
```

> **Watch out:** scanned PDFs contain images, not text, so `extract_text()` returns nothing. They need OCR first. Tables and multi-column layouts often come out scrambled; check a sample before indexing thousands of files.

5. Chunking

Documents are split into **chunks** before embedding, for three reasons: embedding models have an input limit, a small focused chunk produces a sharper embedding than a whole document, and you only want to put the *relevant* part in the prompt.

Fixed-size chunks with overlap

The simplest method cuts every N words, with an **overlap** so that a sentence cut at a boundary still appears whole in one of the chunks.

Python — Tested: output below is real

```python
article = ("Brightwave routers support Wi-Fi 6. To change your Wi-Fi password, open the app and "
           "go to Settings, then Network. Choose a password of at least 12 characters. "
           "Your devices will disconnect and must reconnect with the new password. "
           "If you forget the password, hold the reset button for 10 seconds to restore "
           "the factory settings printed on the router label.")

def chunk_words(text, size=20, overlap=5):
    words = text.split()
    step = size - overlap
    return [" ".join(words[i:i + size]) for i in range(0, max(len(words) - overlap, 1), step)]

for n, c in enumerate(chunk_words(article), start=1):
    print(f"[{n}] {c}\n")
```

Output

```
[1] Brightwave routers support Wi-Fi 6. To change your Wi-Fi password, open the app and go to Settings,
then Network. Choose

[2] to Settings, then Network. Choose a password of at least 12 characters. Your devices will disconnect
and must reconnect with

[3] disconnect and must reconnect with the new password. If you forget the password, hold the reset button
for 10 seconds

[4] reset button for 10 seconds to restore the factory settings printed on the router label.
```

Splitting on natural boundaries

Fixed-size chunks cut sentences in half. A better approach is **recursive splitting**: try to split on paragraphs first, then sentences, then words, merging small pieces together until each chunk is close to the target size. This is what LangChain's `RecursiveCharacterTextSplitter` does.

Python — Tested: output below is real

```python
def chunk_sentences(text, max_words=30):
    sentences = re.split(r"(?<=[.!?])\s+", text.strip())
    chunks, current = [], []
    for sentence in sentences:
        if current and len(" ".join(current + [sentence]).split()) > max_words:
            chunks.append(" ".join(current))
            current = []
        current.append(sentence)
    if current:
        chunks.append(" ".join(current))
    return chunks

for n, c in enumerate(chunk_sentences(article), start=1):
    print(f"[{n}] {c}\n")
```

Output

```
[1] Brightwave routers support Wi-Fi 6. To change your Wi-Fi password, open the app and go to Settings,
then Network. Choose a password of at least 12 characters.

[2] Your devices will disconnect and must reconnect with the new password.

[3] If you forget the password, hold the reset button for 10 seconds to restore the factory settings
printed on the router label.
```

Every chunk now contains whole sentences, and each one is about a single topic.

Choosing a chunk size

Chunk size	Strengths	Weaknesses
Small (100–200 tokens)	Precise retrieval; little irrelevant text in the prompt	May lack the context needed to answer
Medium (300–600 tokens)	A good default for most documents	
Large (800+ tokens)	Keeps context together	Blurry embeddings; fewer chunks fit in the prompt

Start around 300–500 tokens with 10–20% overlap, then **measure** (Section 9). Two more techniques are worth knowing:

- **Add context to each chunk.** Prepend the document title and section heading ("Brightwave Router Guide > Wi-Fi > Change password: ..."), so a chunk still makes sense on its own. Anthropic's *contextual retrieval* goes further: it uses an LLM to write a one-sentence summary of where each chunk fits in its document.
- **Semantic chunking.** Embed each sentence and start a new chunk where the meaning shifts, which you detect as a drop in similarity between neighbouring sentences.

6. Embedding and storing

For semantic retrieval, each chunk is embedded and stored in a vector database along with its text and metadata. (The Embeddings guide covers models and vector databases in depth.)

Python — Needs: pip install chromadb openai

```python
import os
import chromadb
from chromadb.utils.embedding_functions import OpenAIEmbeddingFunction

embedder = OpenAIEmbeddingFunction(api_key=os.environ["OPENAI_API_KEY"],
                                   model_name="text-embedding-3-small")
client = chromadb.PersistentClient(path="./brightwave_db")
collection = client.get_or_create_collection("help_centre", embedding_function=embedder)

collection.add(
    ids=[f"doc-{i}" for i in range(len(knowledge_base))],
    documents=knowledge_base,
    metadatas=[{"source": "help_centre", "article": i} for i in range(len(knowledge_base))],
)
print(collection.count(), "chunks indexed")
```

7. Prompting for grounded answers

The prompt is where you turn retrieval into a trustworthy answer. Good RAG prompts:

1. **Restrict the model to the sources**: "Answer using ONLY the sources below."
2. **Allow "I don't know"**, so the model doesn't invent an answer when retrieval fails.
3. **Require citations** with numbered sources, so users can check the evidence.

4. **Put the question after the context.** Models attend best to the start and end of long prompts; information in the middle is used less reliably (the "lost in the middle" effect). Put the most relevant chunks first.
5. **Treat retrieved text as data, not instructions** (see Section 17).

8. A full RAG app: OpenAI + Chroma

Python — Needs: pip install chromadb openai

```python
import os
import chromadb
from chromadb.utils.embedding_functions import OpenAIEmbeddingFunction
from openai import OpenAI

llm = OpenAI()
embedder = OpenAIEmbeddingFunction(api_key=os.environ["OPENAI_API_KEY"], model_name="text-embedding-3-small")
collection = chromadb.PersistentClient(path="./brightwave_db").get_or_create_collection(
    "help_centre", embedding_function=embedder)

PROMPT = """You are Brightwave's support assistant.
Answer using ONLY the sources below and cite them like [1].
If the answer isn't in the sources, say "I don't know".

Sources:
{sources}

Question: {question}"""

def ask(question, k=4):
    hits = collection.query(query_texts=[question], n_results=k)
    chunks = hits["documents"][0]
    sources = "\n".join(f"[{n}] {text}" for n, text in enumerate(chunks, start=1))
    response = llm.chat.completions.create(
        model="gpt-4.1", temperature=0,
        messages=[{"role": "user", "content": PROMPT.format(sources=sources, question=question)}])
    return response.choices[0].message.content, chunks

answer, evidence = ask("My modem broke, how fast can I get a new one?")
print(answer)
```

Part 3: Improving retrieval

> **Key idea:** if the right chunk isn't retrieved, even the best LLM can't answer correctly. Most RAG failures are **retrieval** failures, so measure retrieval first.

9. Measure first: a retrieval test set

Write down realistic questions and which document should answer each one. Real users rarely use your documents' exact words, so include paraphrases. Then measure **Recall@k** (is the right document in the top *k*?) and **MRR** (mean reciprocal rank: how high is it?).

Python — Tested: output below is real

```python
test_set = [   # (question, index of the document that answers it)
    ("E-4021", 2),                              ("what does error E-1007 mean", 3),
    ("how much is the fibre 1000 package", 6),  ("my wifi is really slow", 4),
    ("how do I get my money back", 0),          ("I want to end my contract", 1),
    ("phone number for help", 7),               ("relocating to another home", 5),
    ("stop my kids seeing certain sites", 8),   ("when will I get my invoice", 9),
    ("my modem broke", 10),                     ("when does the technician come", 11),
]

def rank_of(scores, correct):
    # Pessimistic rank: count every document scoring at least as high as the correct one.
    # Without this, a query that scores every document 0 could "find" the answer by luck.
    return int((scores >= scores[correct]).sum())

def evaluate(search_fn, label):
    ranks = []
    for question, correct in test_set:
        ranks.append(rank_of(search_fn(question), correct))
    ranks = np.array(ranks)
    print(f"{label}: Recall@1={np.mean(ranks <= 1):.2f}  Recall@3={np.mean(ranks <= 3):.2f}  MRR=
{np.mean(1 / ranks):.2f}")
    return ranks

ranks = evaluate(retriever.scores, "BM25")
print()
for (question, _), rank in zip(test_set, ranks):
    print(f"  rank {rank:2d}  {'ok  ' if rank <= 3 else 'MISS'}  {question}")
```

Output

```
BM25: Recall@1=0.42  Recall@3=0.42  MRR=0.47

  rank  1  ok    E-4021
  rank  1  ok    what does error E-1007 mean
  rank  1  ok    how much is the fibre 1000 package
  rank  1  ok    my wifi is really slow
  rank 12  MISS  how do I get my money back
  rank  1  ok    I want to end my contract
  rank 12  MISS  phone number for help
  rank 12  MISS  relocating to another home
  rank 12  MISS  stop my kids seeing certain sites
  rank 12  MISS  when will I get my invoice
  rank 12  MISS  my modem broke
  rank 12  MISS  when does the technician come
```

10. The vocabulary mismatch problem

The results split cleanly. BM25 is **perfect** when the question shares words with the answer, especially rare ones like *E-4021*. It **fails completely** when the user's words differ from the document's: "money back" versus "refund", "help" versus "support", "modem" versus "router", "technician" versus "engineer", "kids seeing sites" versus "parental controls", "invoice" versus "bills".

A rank of 12 means BM25 scored every document 0: the question shares no searchable word with any of them. This is the **vocabulary mismatch problem**, and it's the most common reason RAG systems give wrong answers. There are two main fixes: **search by meaning** (dense embeddings, combined with keywords in *hybrid search*), and **rewrite the question** into the documents' vocabulary.

11. Hybrid search

Dense embeddings match meaning ("modem" ≈ "router") but can blur exact tokens such as error codes, product IDs and names. BM25 is the opposite. **Hybrid search** runs both and merges the results.

The standard way to merge is **Reciprocal Rank Fusion (RRF)**. Each document scores 1 / (k + rank) in each ranking, with k = 60 by convention, and the scores are summed. RRF uses only *ranks*, so it doesn't matter that BM25 scores and cosine similarities are on different scales.

Python — Tested: output below is real

```python
def reciprocal_rank_fusion(rankings, k=60):
    scores = {}
    for ranking in rankings:
        for rank, doc_id in enumerate(ranking, start=1):
            scores[doc_id] = scores.get(doc_id, 0) + 1 / (k + rank)
    return sorted(scores, key=scores.get, reverse=True)

# Two retrievers disagree about the order:
keyword_ranking = ["refund policy", "cancellation", "billing", "router replacement"]
semantic_ranking = ["router replacement", "refund policy", "installation", "billing"]
print(reciprocal_rank_fusion([keyword_ranking, semantic_ranking]))
```

Output

```
['refund policy', 'router replacement', 'billing', 'cancellation', 'installation']
```

Documents ranked well by *both* rise to the top; a document ranked first by only one retriever still scores well. Here is hybrid search with a real embedding model, plugged into the same evaluation:

Python — Needs: pip install sentence-transformers

```python
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")
doc_vectors = model.encode(knowledge_base, normalize_embeddings=True)

def dense_scores(question):
    return doc_vectors @ model.encode(question, normalize_embeddings=True)

def hybrid_scores(question, k=60):
    fused = np.zeros(len(knowledge_base))
    for scores in (retriever.scores(question), dense_scores(question)):
        for rank, doc_id in enumerate(np.argsort(-scores), start=1):
            fused[doc_id] += 1 / (k + rank)
    return fused

evaluate(retriever.scores, "BM25  ")
evaluate(dense_scores, "Dense ")
evaluate(hybrid_scores, "Hybrid")
# Expect dense to fix most paraphrase misses, and hybrid to keep BM25's exact-code wins.
```

12. Reranking with a cross-encoder

The retrievers so far are **bi-encoders**: the question and each document are embedded *separately*, which is fast because document vectors are pre-computed. A **cross-encoder** reads the question and a document *together* in one Transformer pass, so attention can compare every word of one with every word of the other. It's much more accurate, but too slow to run on millions of documents.

The standard pattern is **retrieve, then rerank**: fetch the top 20–50 candidates cheaply, then rerank them with a cross-encoder and keep the best 3–5.

Python — Needs: pip install sentence-transformers

```python
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def retrieve_and_rerank(question, fetch=20, keep=3):
    candidates = np.argsort(-hybrid_scores(question))[:fetch]
    pairs = [(question, knowledge_base[i]) for i in candidates]
    scores = reranker.predict(pairs)                    # one relevance score per pair
    best = candidates[np.argsort(-scores)[:keep]]
    return [knowledge_base[i] for i in best]

print(retrieve_and_rerank("my modem broke"))
```

Hosted rerankers (Cohere Rerank, Voyage, Jina) do the same job with an API call.

13. Query rewriting

Users write short, vague or conversational questions. An LLM can **rewrite** them into the vocabulary your documents use before searching. To see why this works, here are rewrites of the kind an LLM produces, run through the same BM25 retriever that failed on the originals:

Python — Tested: output below is real

```python
rewrites = {   # original question -> rewrite in the documents' vocabulary
    "how do I get my money back": "request a refund",
    "phone number for help": "call customer support",
    "my modem broke": "router lost or damaged replacement",
    "when does the technician come": "engineer visit installation",
    "stop my kids seeing certain sites": "parental controls block websites",
    "when will I get my invoice": "when are bills issued",
    "relocating to another home": "moving house transfer broadband",
}
for original, rewritten in rewrites.items():
    correct = dict(test_set)[original]
    before = rank_of(retriever.scores(original), correct)
    after = rank_of(retriever.scores(rewritten), correct)
    print(f"rank {before:2d} -> {after}   {original!r} -> {rewritten!r}")
```

Output

```
rank 12 -> 1   'how do I get my money back' -> 'request a refund'
rank 12 -> 1   'phone number for help' -> 'call customer support'
rank 12 -> 1   'my modem broke' -> 'router lost or damaged replacement'
rank 12 -> 1   'when does the technician come' -> 'engineer visit installation'
rank 12 -> 1   'stop my kids seeing certain sites' -> 'parental controls block websites'
rank 12 -> 1   'when will I get my invoice' -> 'when are bills issued'
rank 12 -> 1   'relocating to another home' -> 'moving house transfer broadband'
```

Every failed query now finds the right document first. In a real system, the LLM writes the rewrite:

Python — Needs: pip install openai

```python
def rewrite_query(question, history=""):
    prompt = f"""Rewrite the user's question as a short search query for a broadband
company's help centre. Use the formal terms a help article would use
(for example: router, engineer, parental controls, bills).
If there is conversation history, make the query self-contained.

History: {history}
Question: {question}
Search query:"""
    response = client.chat.completions.create(
        model="gpt-4.1", temperature=0, messages=[{"role": "user", "content": prompt}])
    return response.choices[0].message.content.strip()

print(rewrite_query("and how much does that cost?", history="User asked about Fibre 1000."))
# "Fibre 1000 plan monthly price"
```

Two related techniques:

- **Multi-query retrieval**: generate three or four different phrasings, retrieve for each, and merge the results with RRF.
- **Conversational rewriting**: in a chat, "how much does that cost?" means nothing alone. Rewrite it using the conversation history first, as in the example above.

14. HyDE: search with a hypothetical answer

HyDE (Hypothetical Document Embeddings) asks the LLM to *write a plausible answer first*, even if the details are wrong, then searches with the embedding of that answer. A fake answer is written in the same style and vocabulary as the real documents, so it lands closer to them in embedding space than a short question does.

Python — Needs: pip install openai sentence-transformers

```python
def hyde_search(question, k=3):
    fake_answer = client.chat.completions.create(
        model="gpt-4.1", temperature=0,
        messages=[{"role": "user", "content": f"Write a short help-centre article answering:
{question}"}],
    ).choices[0].message.content
    query_vector = model.encode(fake_answer, normalize_embeddings=True)
    return [knowledge_base[i] for i in np.argsort(-(doc_vectors @ query_vector))[:k]]
```

> **Watch out:** every LLM call before retrieval adds latency and cost. Use rewriting and HyDE where your evaluation shows they help, not by default.

15. Other retrieval techniques

Technique	What it does	When to use it
Metadata filtering	Restrict search by date, product, language or user permissions	Almost always; essential for access control
Parent-document (small-to-big)	Search small chunks for precision, but send the larger parent section to the LLM	When answers need surrounding context
Contextual compression	Trim each retrieved chunk to the sentences relevant to the question	When chunks are long and prompt space is tight
Maximal marginal relevance (MMR)	Pick results that are relevant *and* different from each other	When the top results are near-duplicates

Part 4: Advanced and production

16. Evaluating the answers

Retrieval metrics tell you whether the right evidence was found. You also need to know whether the **answer** is good. The standard checks are:

- **Faithfulness (groundedness)**: is every claim in the answer supported by the retrieved sources? This catches hallucinations.
- **Answer relevance**: does the answer actually address the question?
- **Context relevance**: were the retrieved chunks useful, or mostly noise?
- **Correctness**: does the answer match a reference answer, when you have one?

These are usually scored by an **LLM-as-judge**: a second model call with a strict rubric.

Python — Needs: pip install openai

```python
import json

JUDGE = """You are grading a support bot's answer.
Sources:
{sources}

Answer:
{answer}

For each claim in the answer, check whether the sources support it.
Return JSON: {{"supported_claims": int, "unsupported_claims": int, "faithful": true/false, "reason":
"..."}}"""

def judge_faithfulness(answer, sources):
    response = client.chat.completions.create(
        model="gpt-4.1", temperature=0, response_format={"type": "json_object"},
        messages=[{"role": "user", "content": JUDGE.format(sources="\n".join(sources),
answer=answer)}])
    return json.loads(response.choices[0].message.content)
```

Libraries such as **RAGAS**, **DeepEval** and **TruLens** package these metrics. Whichever you use, check a sample of the judge's verdicts by hand. Judges make mistakes too.

17. RAG with LangChain

Frameworks save boilerplate: loaders for many file types, splitters, vector store integrations and tracing. Here is the same pipeline with LangChain v1:

Python Needs: pip install langchain langchain-openai langchain-chroma langchain-community langchain-text-splitters pypdf

```python
from langchain_chroma import Chroma
from langchain_community.document_loaders import PyPDFLoader
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Indexing
pages = PyPDFLoader("employee_handbook.pdf").load()          # one Document per page, with metadata
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=150)
chunks = splitter.split_documents(pages)
vector_store = Chroma.from_documents(chunks, OpenAIEmbeddings(model="text-embedding-3-small"),
                                     persist_directory="./handbook_db")
retriever = vector_store.as_retriever(search_kwargs={"k": 4})

# Querying
llm = ChatOpenAI(model="gpt-4.1", temperature=0)

def ask(question):
    docs = retriever.invoke(question)
    sources = "\n\n".join(f"[{n}] (page {d.metadata.get('page', 0) + 1}) {d.page_content}"
                          for n, d in enumerate(docs, start=1))
    prompt = (f"Answer using only these sources and cite them like [1]. "
              f"If the answer isn't there, say you don't know.\n\n{sources}\n\nQuestion: {question}")
    return llm.invoke(prompt).content

print(ask("How many days of annual leave do new employees get?"))
```

18. Agentic RAG

Classic RAG retrieves exactly once, for every question. **Agentic RAG** gives an LLM agent a *search tool* and lets it decide **whether** to search, **what** to search for, and whether to **search again** after reading the results. This handles questions that need several lookups ("Compare the cancellation fee with the cost of a replacement router") or no lookup at all ("thanks!").

Python — Needs: pip install langchain langchain-openai langchain-chroma

```python
from langchain.agents import create_agent
from langchain.tools import tool
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings

help_centre = Chroma.from_texts(knowledge_base, OpenAIEmbeddings(model="text-embedding-3-small"))
retriever = help_centre.as_retriever(search_kwargs={"k": 3})

@tool
def search_help_centre(query: str) -> str:
    """Search Brightwave's help centre. Use precise terms such as router, bills, engineer."""
    hits = retriever.invoke(query)
    return "\n\n".join(d.page_content for d in hits)

agent = create_agent(
    model="openai:gpt-4.1",
    tools=[search_help_centre],
    system_prompt="You are Brightwave's support assistant. Search the help centre before "
                  "answering factual questions. Search again with different terms if the "
                  "first results don't answer the question. Never invent policies.",
)

result = agent.invoke({"messages": [{"role": "user", "content":
    "If I cancel in month 6, is that cheaper than paying for a replacement router?"}]})
print(result["messages"][-1].content)
```

The trade-off: agents are more capable, but slower, more expensive and less predictable. Use them when questions genuinely need multi-step reasoning.

19. Other advanced patterns

- **GraphRAG**: extract entities and relationships from documents into a knowledge graph, then retrieve along connections. This helps "global" questions like "What are the main themes across all 5,000 support tickets?", which no single chunk can answer.
- **Multimodal RAG**: index images, charts and scanned pages using multimodal embeddings or vision models, so answers can draw on diagrams and tables.
- **RAG versus long context**: modern models accept hundreds of thousands of tokens. For a few small documents, putting everything in the prompt can beat RAG. RAG still wins on large or changing collections, cost, latency, citations and access control.

20. Production checklist

Area	What to get right
Security	Retrieved documents can contain malicious instructions ("ignore previous instructions and..."). This is **indirect prompt injection**. Treat retrieved text as data: delimit it clearly, never let it trigger tools or actions without checks, and sanitise untrusted sources.
Access control	Filter retrieval by the user's permissions *before* anything reaches the LLM. Never rely on the prompt to hide data.
Freshness	Re-index when documents change; store timestamps; delete chunks from removed documents.
Citations	Show users the sources, ideally with links to the exact page or section.
Evaluation	Keep a test set; re-run it after every change to chunking, models or prompts.
Monitoring	Log questions, retrieved chunks, answers and user feedback. Questions that get "I don't know" show you which documents are missing.
Cost and latency	Cache frequent answers and embeddings; use smaller models where your evaluation allows.

Summary and interview questions

Cheat sheet

Concept	One-line summary
RAG	Retrieve relevant text, put it in the prompt, generate a grounded answer
Indexing	Load, chunk, embed and store documents once, ahead of time
Chunking	Split documents into focused, self-contained passages
BM25	Keyword ranking; excellent for exact terms, codes and names
Dense retrieval	Embedding similarity; matches meaning across different words
Vocabulary mismatch	Users and documents use different words for the same thing
Hybrid search + RRF	Combine keyword and dense rankings by rank position
Cross-encoder reranking	Score question–document pairs jointly; accurate but slow, so used on a shortlist
Query rewriting, HyDE	Use an LLM to bridge the gap between questions and documents
Faithfulness	Is every claim supported by the retrieved sources?
Agentic RAG	An agent decides when and what to search, possibly several times
Indirect prompt injection	Malicious instructions hidden in retrieved documents

Interview questions to practise

1. **Walk me through a RAG system.** Indexing: load, chunk, embed, store with metadata. Querying: embed the question, retrieve top-k (ideally hybrid plus reranking), build a grounded prompt with numbered sources, generate with citations, allow "I don't know".
2. **Your RAG system gives wrong answers. How do you debug it?** Separate retrieval from generation. First check whether the right chunk was retrieved (recall@k on a test set). If not, fix chunking, retrieval or queries. If it was retrieved but the answer is still wrong, fix the prompt, the chunk order or the model.
3. **When would you choose RAG over fine-tuning?** When the knowledge changes, must be cited, or is permission-restricted. Fine-tune to change behaviour or format, not to add facts.
4. **Why use hybrid search?** Dense retrieval handles paraphrases; BM25 handles exact terms like codes and names. RRF combines them without needing comparable scores.
5. **What is a cross-encoder and why not use it for all retrieval?** It processes the question and document together, which is more accurate, but it can't pre-compute document vectors, so it's too slow for a full collection. Use it to rerank a shortlist.
6. **How do you evaluate a RAG system?** Retrieval metrics (recall@k, MRR) on a labelled question set, plus answer metrics (faithfulness, relevance, correctness) using LLM-as-judge checked against human review.
7. **What security risks are specific to RAG?** Indirect prompt injection from retrieved content, and data leakage across users. Mitigate with permission filtering before retrieval and by treating retrieved text strictly as data.